https://youtu.be/DJmg8sGNBys

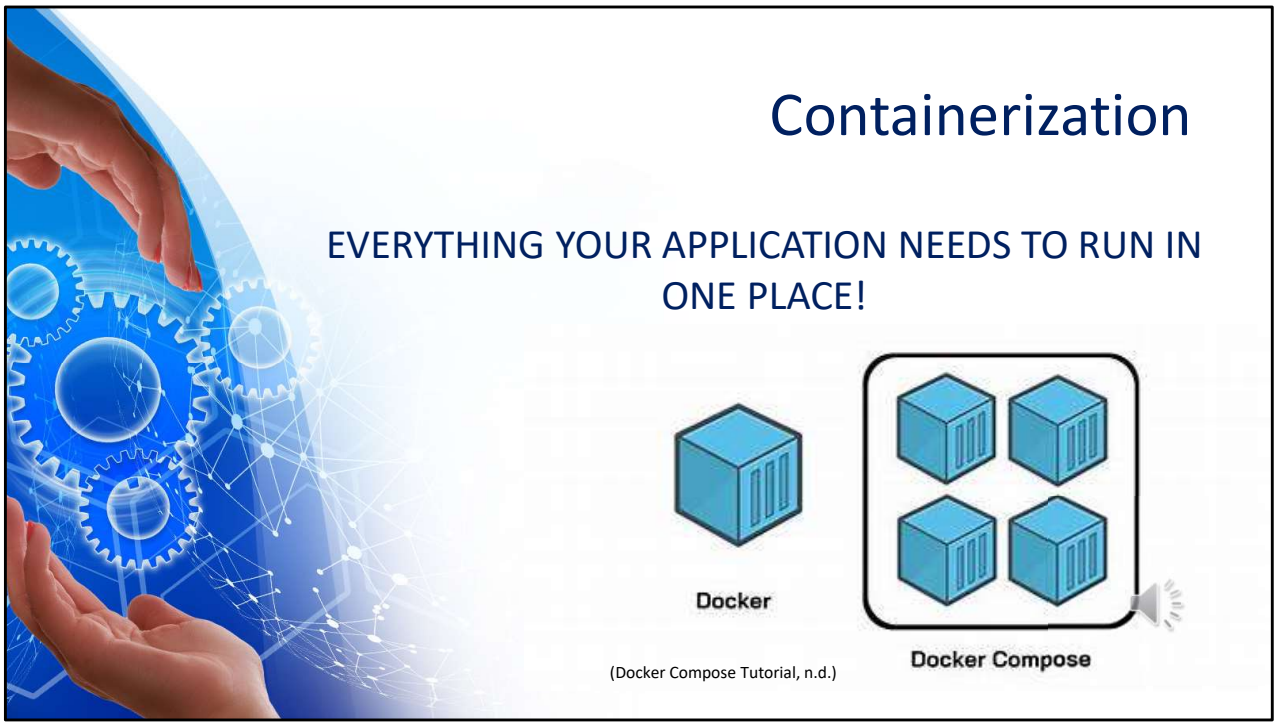
Overview

The Intricacies of Cloud Development:

A Presentation for Both Technical and Nontechnical Audiences

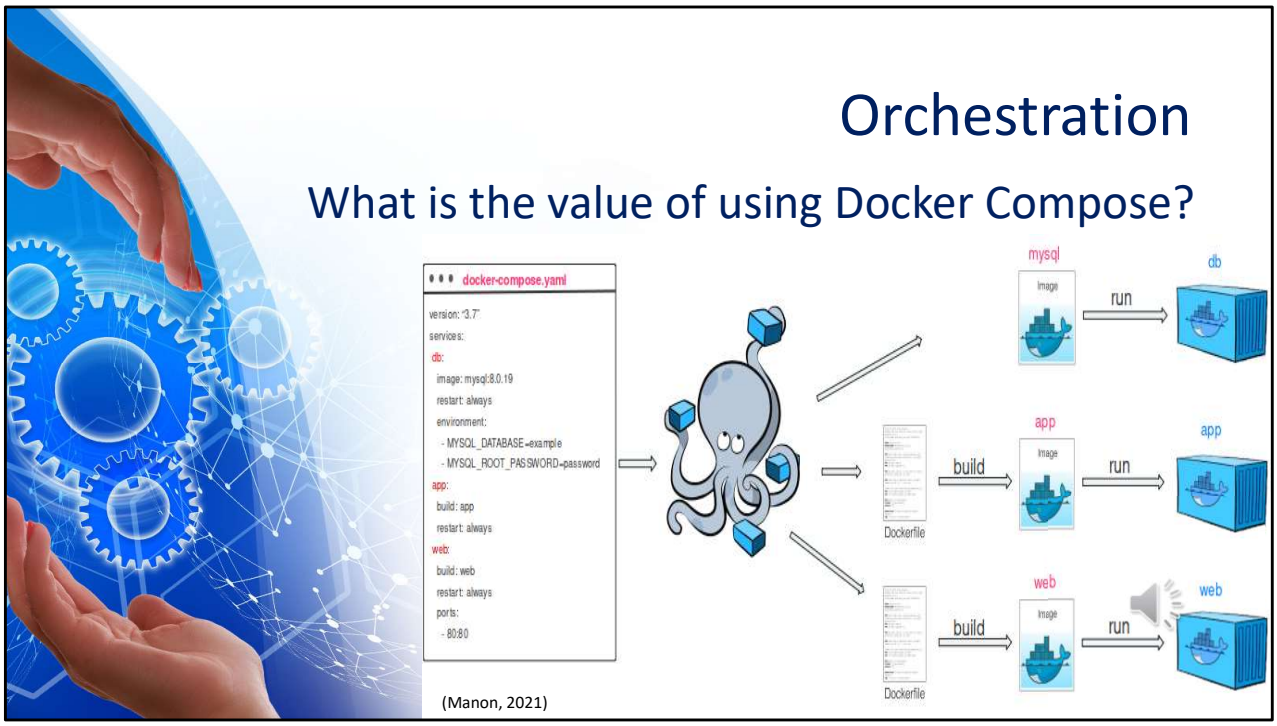
by Kelly Sebastiani

Hello, my name is Kelly Sebastiani, and I am here to share my knowledge, as a new developer, migrating a full stack application to a serverless AWS environment. We will visit the topics on containerization and orchestration through Docker and Docker Compose and AWS based topics on the serverless cloud, cloud-based development principles, and securing your cloud application. The goal of this presentation is to break down the process of cloud migration for both developers and non-developers, highlighting the key benefits and challenges involved.



What is Containerization?
Containerization is a method of packaging an application and everything it needs to run—like the code, libraries, and settings—into a single, self-contained unit called a "container." This container can be easily moved and run on any computer or cloud system without worrying about compatibility issues. It's like putting all the ingredients for a recipe into a box, so you can cook the dish the same way, no matter where you are.

One common tool used to package an application into a container is Docker. If you have multiple containers together as part of one application, like a website and a database, Docker Compose is another tool that can be helpful in this situation. I will give an honorable mention to the tool Kubernetes. Kubernetes helps manage and run containers across many computers. This tool is more complex and more powerful for large-scale and production environments.



Orchestration

What is the value of using Docker Compose?

Docker Compose's value lies in its ability to simplify managing complex applications that need multiple parts working together. Instead of setting up everything manually, you can define how these parts should work in one simple file known as a YAML. This helps ensure everything works the same way across different environments, so you don't run into unexpected issues. It also makes it easy to scale up when more resources are needed.

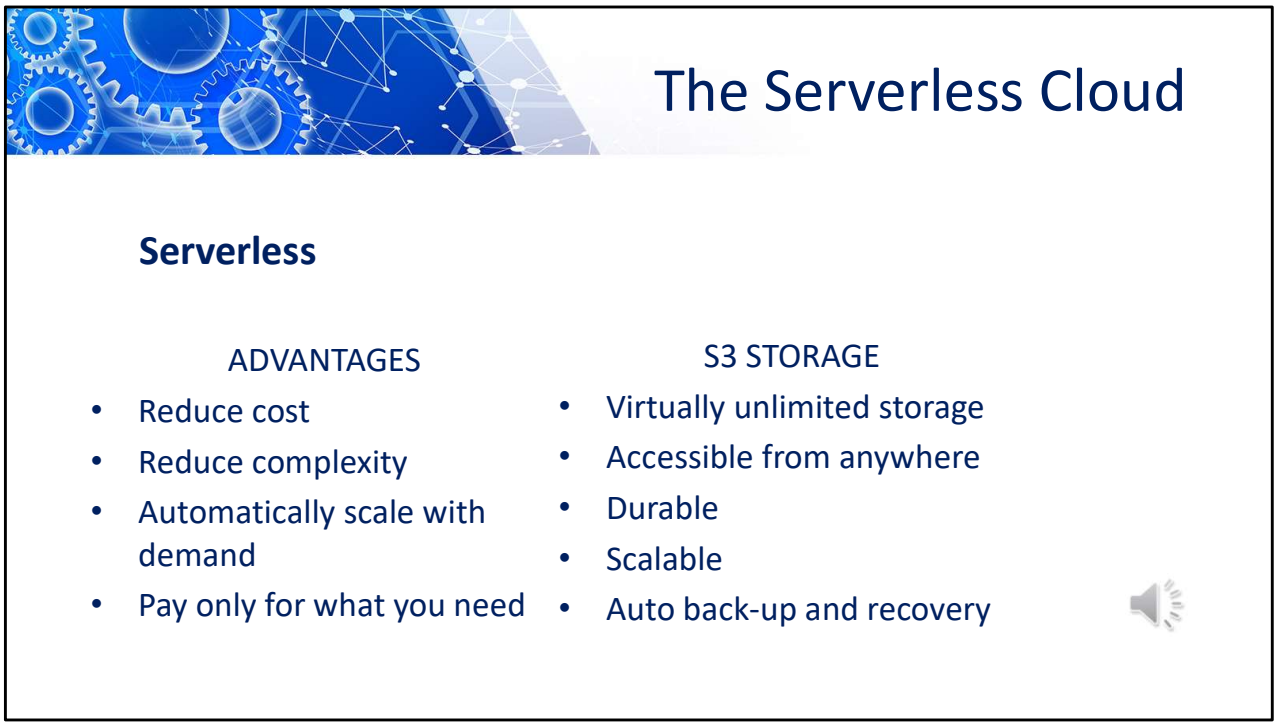
Remember our recipe analogy on containers? Well Docker Compose is like having a meal planner that knows all the dishes you want to make, how to prepare them, and in what order. It organizes everything so that all the dishes are ready at the same time, ensuring the entire meal comes together smoothly.

So now you have your full-stack application, and you want to migrate it to the cloud. AWS provides a cloud adoption framework for this known as the 6 R’s. These models are rehosting, replatforming, repurchasing, refactoring, retire, and retain. Lets break each of them down to understand them a little bit better.

Rehosting, or lift-and-shift, is like moving your office to a new building without changing anything—you simply move your applications to the cloud as they are. Replatforming, or lift-and-reshape, involves making a few tweaks to your applications, like rearranging furniture in a new office, to take advantage of some cloud features.

Repurchasing, or drop and shop, is switching to a new cloud-based service that replaces your old application, much like renting a fully furnished office. Refactoring, or re-architecting, involves significant changes to your applications, redesigning them to fully leverage cloud capabilities.

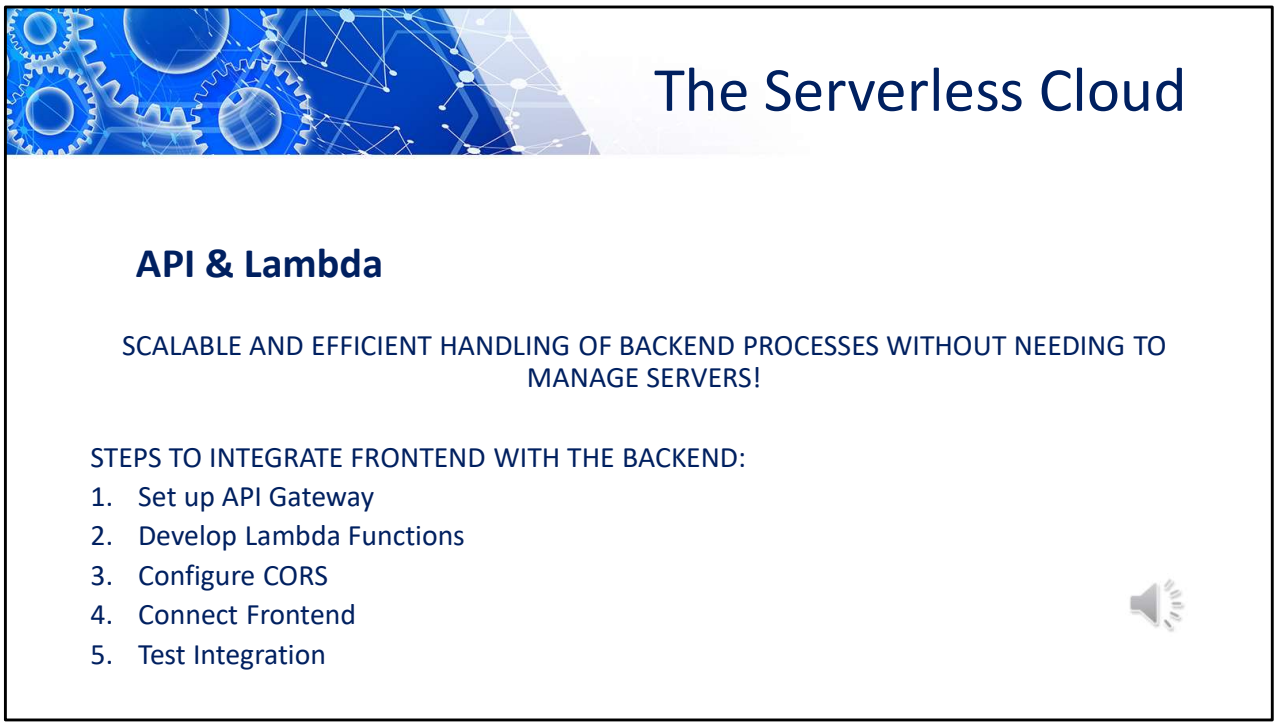
Retire is like clearing out old, unused files before a move—you identify and remove applications that are no longer needed. Retain is deciding to keep some applications in their current environment while moving others, much like leaving some things in your old office while relocating the rest.



Now that you have migrated your full-stack application to the cloud lets talk about what this means.

Serverless computing means that you don't have to manage the servers that run your applications. Instead, cloud providers like AWS automatically handle server management, scaling, and maintenance. Some advantages of this are it reduces operational complexity, automatically scales with demand, and you only pay for the resources you use.

AWS has a cloud-based storage service called S3 which stands for Simple Storage Service. Unlike local storage, which is limited by the physical hardware of a device, S3 provides virtually unlimited storage space that is accessible from anywhere via the internet. It is highly durable, scalable, and allows for automated backup and recovery, making it more reliable and flexible than traditional local storage.



AWS Lambda is a serverless compute service that lets you run code in response to events, such as HTTP requests. The logic works by defining an API through API Gateway, which triggers Lambda functions when specific endpoints are hit. The function executes your business logic, processes the request, and returns a response. This allows for scalable and efficient handling of backend processes without needing to manage servers.

To implement this serverless architecture, you typically write infrastructure-as-code scripts that define the setup of API Gateway, Lambda functions, and S3 buckets. Additionally, you'll create Lambda function scripts in languages like Python, Node.js, or JavaScript to handle the backend logic, and frontend scripts to interact with the APIs.

To integrate the frontend with the backend, take the following steps:

1. Set up API Gateway: Create endpoints that the frontend can call.
2. Develop Lambda Functions: Write the backend logic to process requests.
3. Configure CORS: Ensure cross-origin requests are allowed from the frontend.
4. Connect Frontend: Update the frontend code to send HTTP requests to the API Gateway endpoints.
5. Test Integration: Ensure the frontend and backend communicate correctly and handle any errors.

The Serverless Cloud

Database

MongoDB

- Document-based
- Schema-less
- Scalable
- Indexing
- Rich-query language
- Infrastructure dependent cost
- Broad integration

DynamoDB

- Key-value and document-based
- Defined-schema for primary keys
- Natively scalable
- Low-latency performance
- Primary and secondary indexing
- Pay-per-use cost model
- Integration within AWS

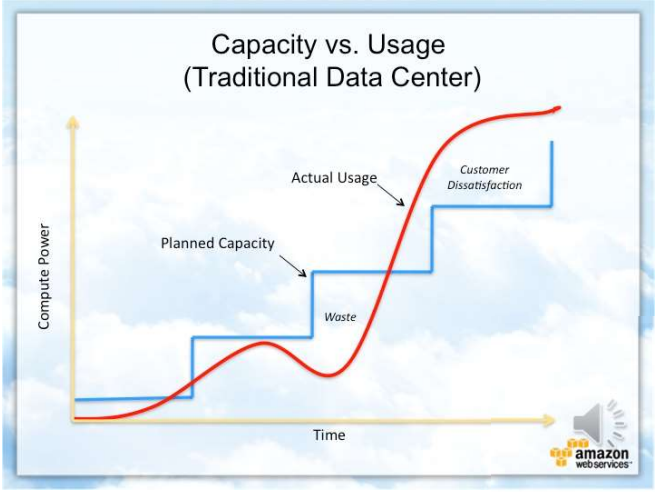
To address databases, we will specifically be looking at MongoDB and AWS's database, DynamoDB…

- MongoDB offers flexible, schema-less storage, ideal for diverse data models, while DynamoDB provides a more structured key-value and document model optimized for performance.
- Both scale horizontally, but DynamoDB excels with seamless, automatic scaling within the AWS ecosystem, perfect for dynamic workloads.
- MongoDB is strong in read-heavy tasks with rich query capabilities, while DynamoDB delivers predictable, low-latency performance, especially for write-heavy operations.
- In indexing, MongoDB is versatile, whereas DynamoDB focuses on primary and secondary indexes for fast lookups.
- Costs differ: MongoDB depends on infrastructure, while DynamoDB's pay-per-use model suits variable workloads.
- Finally, MongoDB integrates broadly, but DynamoDB's tight AWS integration makes it ideal for serverless applications.

These attributes should help guide your choice based on your application's needs.

Cloud-Based Development Principles

- Elasticity – Auto, by demand, scaling
- Pay-for-use model – Pay only for the resources used

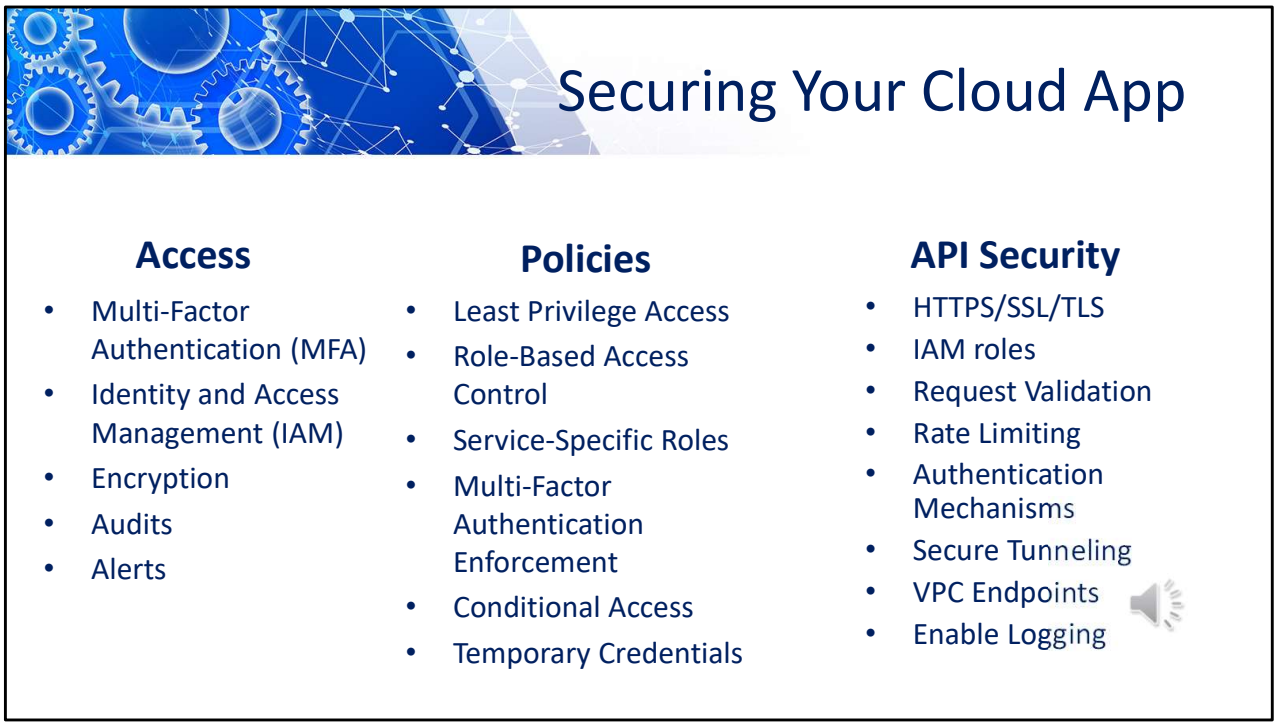


Cloud-Based Development Principles…

Elasticity refers to the cloud's ability to automatically scale resources up or down based on demand. For example, if your application experiences a spike in traffic, the cloud can instantly allocate more resources to handle the load, and then reduce them when traffic decreases. This ensures optimal performance without overpaying for unused resources.

The pay-for-use model means you only pay for the cloud resources you actually consume, such as compute time, storage, or data transfer. Unlike traditional fixed-cost models, this allows for more efficient spending since costs directly correlate with usage, making it ideal for dynamic workloads.

If you remember our earlier slide with advantages to the serverless cloud and S3 storage, these development principles align perfectly.



With the cyber threat landscape continually increasing, security is a growing concern. So how can you secure your cloud application?...
To prevent unauthorized access in your cloud environment, it's crucial to implement strong authentication and authorization mechanisms. This includes using Multi-Factor Authentication, managing access through Identity and Access Management, and encrypting sensitive data both at rest and in transit. Regularly auditing access logs and setting up alerts for unusual activity can further bolster your security measures.

In managing permissions, roles and policies work hand-in-hand. Roles define what users, services, or applications can do, while policies explicitly outline those permissions. By attaching policies to roles, you can precisely control access within your cloud application. Custom policies can be crafted to meet specific needs, such as allowing a Lambda function to read from an S3 bucket without write access. This approach minimizes the risk of unauthorized actions.

Securing connections between Lambda and other AWS services involves enabling HTTPS on API Gateway, using IAM roles for access control, and implementing request validation, rate limiting, and authentication. For Lambda's database connections, it's essential to use SSL/TLS encryption and restrict access with IAM roles, utilizing VPC endpoints or secure tunneling to keep traffic within the AWS network.

Finally, for S3, enforcing least privilege policies, enabling encryption for data at rest and in transit, and monitoring access through logging are key steps to ensure that only authorized actions take place.

CONCLUSION

- Containerization packages app needs in one place and Docker is a good tool for this.
- Docker-compose is a great tool to orchestrate multiple containers in a single app.
- Migrate your full-stack app to AWS's serverless cloud using one of the 6 R's.
- Serverless computing conveniently manages servers for you.
- Address security concerns in your cloud application through roles, policies, and various strategies preventing unauthorized access.

Thank you for your time.

In conclusion, to effectively manage and deploy your applications, start by using Docker for containerization, which packages everything your app needs to run in one place. For orchestrating multiple containers within a single app, Docker-compose is an excellent tool. When migrating your full-stack application to AWS's serverless cloud, consider using one of the 6 R's to find the best strategy for your needs. Serverless computing offers the convenience of automatic server management, freeing you from the complexities of infrastructure. Finally, ensure your cloud application is secure by implementing roles, policies, and strategies to prevent unauthorized access.

CITATIONS

Docker-compose – Manon Biaudelle. Docker Compose Orchestration Image. (2021, July 23). Biaudelle.fr. https://www.biaudelle.fr/docker-compose/

Docker Compose Tutorial: advanced Docker made simple. (n.d.). Educative: Interactive Courses for Software Developers. https://www.educative.io/blog/docker-compose-tutorial